Distributed systems: events and messages

A distributed system has independent processes that communicate by messages.

An event is an action observed within one process: a computation, send or receive.

Each process has a local sequence of events. There is no shared global clock.

A message is sent at one event and received later by another process.

Example: process A sends an order to B. B receives it and records the order.

Local clocks may disagree; timestamps alone do not prove causal order.

Causal ordering: the happens-before relation

If two events occur in one process, the earlier event happens before the later one.

Sending a message happens before receiving that message.

The relation is transitive: if a precedes b and b precedes c, a precedes c.

Events are concurrent when neither event happens before the other.

Example: A sends an order, B receives it, then B sends a confirmation to A.

The send precedes the receipt, which precedes the confirmation.

Independent local computations in A and C may be concurrent.

Causality gives a partial order; it does not impose a total order on all events.